



# Pengayaan: Keamanan, Hak Akses, RAID & Docker

Week 13: Hak Akses File, Virus dan Security, RAID, serta Container

**[informatika-itera.github.io/os](https://informatika-itera.github.io/os)**

Program Studi Teknik Informatika  
Institut Teknologi Sumatera

2026

# Outline

1. **Hak Akses di Linux**
2. **Virus & Keamanan Sistem Operasi**
3. **Docker & Container**
4. **RAID & Storage Redundancy**

# Capaian Pembelajaran

Tujuan setelah pertemuan selesai

1. Memahami model hak akses file di Linux (rwx, owner/group/others).
2. Mengenal berbagai jenis malware dan bagaimana OS melindungi diri.
3. Memahami konsep RAID dan perbedaan level-levelnya.
4. Menjelaskan perbedaan container dan VM serta konsep dasar Docker.

## Fokus Kelas Ini

Kita fokus pada empat topik pengayaan: **Hak Akses**, **Keamanan OS**, **RAID**, dan **Docker**.

# Hak Akses di Linux

# Mengapa Hak Akses Penting?

Dari Week 12 kita tahu file disimpan – tapi siapa yang boleh mengaksesnya?

## Masalah

- Dalam sistem multi-user, banyak orang bisa pakai komputer yang sama.
- File A milik Budi – jangan sampai Siti bisa baca atau ubah.
- OS harus bisa menjawab: **who can do what to which file?**

## Analogi: Kantor Sewaan

- Setiap karyawan punya ruangan sendiri (home directory).
- Beberapa ruangan bersifat umum (shared folder).
- Ada dokumen rahasia yang hanya boleh dibaca direktur.
- Resepsionis cek kartu akses setiap kali masuk ruangan.

# Model Unix: Owner, Group, Others

Tiga kategori pengguna + tiga jenis izin

## Tiga Kategori

- **Owner (u)** – Pemilik file (biasanya pembuat).
- **Group (g)** – Semua anggota grup yang sama.
- **Others (o)** – Semua pengguna lain di sistem.

## Tiga Jenis Izin

- **r (read)** – Baca isi file / lihat daftar direktori.
- **w (write)** – Ubah file / buat-hapus file di direktori.
- **x (execute)** – Jalankan file sebagai program / masuk ke direktori.

## Contoh: `ls -l`

`-rwxr-xr-`

Owner: rwx (baca-tulis-jalan)

Group: r-x (baca-jalan)

Other: r- (baca saja)

→ Owner bisa edit dan jalankan.

→ Orang lain cuma bisa baca.

# Numeric Notation (Mode Oktal)

Cara cepat menulis permission dengan angka

| Angka | Biner | Izin |
|-------|-------|------|
| 7     | 111   | rwX  |
| 6     | 110   | rw-  |
| 5     | 101   | r-X  |
| 4     | 100   | r--  |
| 0     | 000   | ---  |

## Rumus

```
chmod 754 file =  
Owner: 7 (rwX)  
Group: 5 (r-X)  
Other: 4 (r--)
```

## Contoh Umum

- 755 – script/program (rwxr-xr-x)
- 644 – dokumen biasa (rw-r--r--)
- 600 – file pribadi (rw-----)
- 700 – direktori pribadi (rwx-----)

# Symbolic Notation: `chmod [who] [+/-/=] [perms]`

Mengubah permission dengan simbol, bukan angka

## Format Umum

`chmod [who] [+/-/=] [permissions] file`

- **Who:** u (user), g (group), o (other), a (all).
- **Operator:** + tambah, - hapus, = set persis.
- **Perms:** r, w, x.

## Kombinasi

- `chmod u+x file` – tambah execute untuk user.
- `chmod go-rw file` – hapus read+write untuk group & other.
- `chmod a=r file` – semua cuma bisa baca.

## Contoh Langkah demi Langkah

Awal: `-rwxr-xr-` (mode 754)

1. `chmod u-w`  
`-r-xr-xr-` ← user kehilangan write
2. `chmod 755`  
`-rwxr-xr-x` ← other dapat execute
3. `chmod 644`  
`-rw-r-r-` ← user & group kehilangan execute



# Special Bits: Setuid, Setgid, Sticky Bit

Istimewa – izin di luar rwx biasa

## SUID (Set User ID) – `chmod u+s`

- Program jalan dengan **privilege pemiliknya**, bukan yang menjalankan.
- Contoh: `passwd` – perlu akses root untuk ubah password, tapi user biasa bisa jalankan.
- Tampil sebagai `s` di posisi `x` owner:  
`-rwsr-xr-x`.

## SGID & Sticky Bit

### SGID (`chmod g+s`):

- File: jalan dengan privilege grup pemilik.
- Direktori: file baru mewarisi grup direktori.

### Sticky Bit (`chmod +t`):

- Di `/tmp`: semua bisa tulis, tapi cuma owner yang bisa hapus filenya.
- Tampil sebagai `t`: `drwxrwxrwt`.

# Checkpoint 1

## Hak Akses – Cek pemahaman sebelum lanjut

1. Apa arti `chmod 600 rahasia.txt`?
2. Bedanya SUID dan SGID?
3. Kenapa `/tmp` pakai sticky bit?
4. Konversi `-rwxr-xr-` ke notasi oktal (3 digit).
5. Permission akhir setelah `chmod 755 file.txt`?
6. Permission akhir setelah `chmod u-w file.txt` (awal `-rwxr-xr-`)?

### Jawaban Singkat

1. Owner bisa baca+tulis; group & others tidak bisa apa-apa.
2. SUID = pakai privilege owner file; SGID = pakai privilege grup file.
3. Biar semua user bisa buat file sementara, tapi tidak bisa hapus file punya orang lain.
4. 754 (`rw`=7, `r-x`=5, `r--`=4).
5. `-rwxr-xr-x` (owner: `rw`, group: `r-x`, other: `r-x`).
6. `-r-xr-xr-` (owner kehilangan write).

# **Virus & Keamanan Sistem Operasi**

# Ancaman: Kenapa OS Perlu Aman?

Dunia nyata penuh dengan malware

## Jenis Malware

- **Virus** – Menempel di file lain dan menyebar saat file dijalankan.
- **Worm** – Menyebar sendiri melalui jaringan tanpa perlu file induk.
- **Trojan** – Menyamar sebagai program berguna, tapi diam-diam jahat.
- **Ransomware** – Mengenkripsi file korban, minta tebusan.
- **Rootkit** – Sembunyi di dalam OS, sembunyikan jejak serangan.

## Analogi

- Virus = “flu” komputer – nempel di file.
- Worm = pencuri yang masuk lewat celah pagar dan ngajak teman-temannya.
- Trojan = “kuda Troya” – kelihatan hadiah, isinya tentara.
- Ransomware = penyekap yang ngunci rumahmu dan minta uang.

# Dari Mana Malware Masuk?

## Vektor serangan umum

- **Email phishing** – Lampiran atau link berbahaya.
- **Download software** – Program bajakan atau dari sumber tidak resmi.
- **USB drive** – Autorun malware dari flash disk.
- **Exploit jaringan** – Celah di layanan yang terhubung ke internet.
- **Drive-by download** – Cuma buka website sudah kena.

## Siapa Targetnya?

- Pengguna biasa: ransomware, phishing.
- Server: DDoS, backdoor.
- Infrastruktur kritis: serangan state-sponsored.
- Ponsel: spyware, adware.

## Pesan Penting

Sebagian besar serangan sukses karena **human error**, bukan karena OS lemah.

# Mekanisme Proteksi oleh OS (1/2)

## User Mode / Kernel Mode & Address Space Isolation

### 1. User Mode vs Kernel Mode

- Aplikasi jalan di **user mode** – akses terbatas.
- OS jalan di **kernel mode** – akses penuh ke hardware.
- Malware di user mode tidak bisa sembarangan merusak kernel.

### 2. Address Space Isolation

- Setiap proses punya address space sendiri.
- Proses A tidak bisa baca memory Proses B.
- Mencegah malware “ngintip” data program lain.

# Mekanisme Proteksi oleh OS (2/2)

## ASLR, NX Bit, & Mandatory Access Control

### 3. ASLR & NX Bit

#### **ASLR (Address Space Layout Randomization):**

- Alamat memory diacak setiap proses jalan.
- Malware susah menebak lokasi kode yang akan dieksekusi.

#### **NX Bit (No-Execute):**

- Menandai halaman memory tertentu sebagai *non-executable*.
- Data tidak bisa dijalankan sebagai kode.

### 4. Mandatory Access Control

**SELinux / AppArmor:** kebijakan keamanan yang membatasi program bahkan oleh root.

# Studi Kasus: Bagaimana Virus Menginfeksi System File?

Ilustrasi langkah demi langkah

1. Pengguna download program bajakan (Trojan).
2. Trojan jalan di **user mode** – tidak bisa langsung ubah kernel.
3. Trojan cari celah (*privilege escalation*):
  - Exploit kernel bug yang belum di-*patch*.
  - Atau tipu user untuk masukkan password sudo.
4. Dapat akses root → overwrite file sistem (misal `/bin/ls`).
5. Setiap kali user ketik `ls`, virus ikut jalan.

## Pelajaran

**Update OS** dan **jangan download sembarangan** adalah pertahanan paling efektif!



## Checkpoint 2

### Virus & Security – Cek pemahaman

1. Apa beda virus dan worm?
2. Bagaimana ASLR membantu mencegah serangan?
3. Kenapa user mode vs kernel mode penting untuk keamanan?

### Jawaban Singkat

1. Virus nempel di file induk; worm menyebar sendiri lewat jaringan.
2. ASLR mengacak alamat memory sehingga malware susah menebak lokasi target.
3. Karena malware di user mode tidak bisa langsung mengakses hardware atau memory kernel.

# Docker & Container

# Masalah: “It Works on My Machine”

Mengapa lingkungan pengembangan tidak konsisten?

## Masalah Nyata

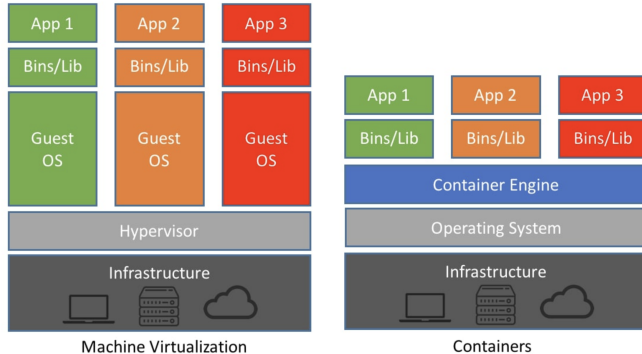
- Develop di Windows, deploy di Linux – beda library.
- Teman pakai Python 3.9, kamu 3.12 – dependency conflicts.
- Setup ulang environment butuh waktu berjam-jam.
- Satu server harus jalan beberapa aplikasi dengan dependency berbeda.

## Analogi: Kontainer Pengiriman

- Bayangkan kontainer kapal barang.
- Isi kontainer bisa apa saja – elektronik, buah, pakaian.
- Tapi kontainernya **standar**: ukuran sama, cara angkut sama.
- Aplikasi + semua dependensinya “dikemas” dalam container.
- Bisa jalan di mana saja tanpa setup ulang!

# Container vs Virtual Machine

Dua pendekatan isolasi yang berbeda



Sumber: NetApp – containers-vs-vms

# Perbandingan Langsung

| Aspek       | Virtual Machine                | Container                     |
|-------------|--------------------------------|-------------------------------|
| Boot time   | Menit                          | Detik                         |
| Ukuran      | GB (OS penuh)                  | MB (aplikasi + lib)           |
| Isolasi     | Kuat (hardware level)          | Level proses (namespace)      |
| Resource    | Boros (setiap VM punya OS)     | Hemat (pakai kernel host)     |
| Portability | Image besar, pindah lambat     | Image kecil, mudah deploy     |
| Use case    | Butuh OS berbeda, multi-tenant | Microservices, CI/CD, dev env |

## Kapan Pilih Mana?

- Butuh jalan aplikasi Linux di Windows server? → VM.
- Butuh deploy 20 microservices di satu server? → Container.

# Docker Basics: Image, Container, Dockerfile

## Tiga konsep inti Docker

### Image

- Template read-only yang berisi aplikasi + semua dependensinya.
- Bisa di-download dari registry (Docker Hub).
- Seperti “CD installer” – tidak bisa diubah.

### Container

- Instance jalan dari sebuah image.
- Bisa dibuat, distart, distop, dihapus.
- Seperti “komputer virtual” mini yang jalan di atas host.

### Dockerfile

- Resep untuk membangun image.
- Berisi perintah: ambil base image, copy file, install, dsb.

### Command Penting

```
docker pull ubuntu
docker run -it ubuntu bash
docker build -t myapp .
docker ps
docker stop <id>
```

# Contoh Dockerfile Sederhana

## Web server statis dengan Nginx

```
# Ambil base image resmi Nginx
FROM nginx:alpine

# Copy file HTML ke direktori default Nginx
COPY index.html /usr/share/nginx/html/

# Ekspos port 80
EXPOSE 80

# Perintah jalan saat container start
CMD ["nginx", "-g", "daemon off;"]
```

### Penjelasan

- FROM – base image yang sudah ada OS + Nginx.
- COPY – masukkan file kita ke dalam image.
- EXPOSE – kasih tahu container ini buka port 80.
- CMD – perintah yang jalan saat container start.

### Build & Run

```
$ docker build -t my-nginx .
$ docker run -d -p 8080:80 my-nginx
```

build image  
jalan container

# Bagaimana Container Isolasi?

## Namespaces dan Cgroups – “Dua Pilar” Docker

### Namespaces (Pemisahan)

Setiap container punya “tampilan” sendiri terhadap sistem:

- `PID ns` – hanya lihat proses di dalam container.
- `NET ns` – network interface sendiri, IP sendiri.
- `MNT ns` – file system root sendiri.
- `UTS ns` – hostname sendiri.
- `USER ns` – user ID sendiri (bisa root di dalam).

### Cgroups (Batas Sumber Daya)

- Membatasi CPU, memory, disk I/O per container.
- Satu container tidak bisa “makan” semua resource.
- Contoh:  
`docker run -memory=256m -cpus=1.5 ...`

### Intuisi: Container = Proses + “Kacamata Kuda”

Container adalah proses biasa, tapi OS memakai namespace untuk membuatnya “lihat” dunia yang terisolasi – seolah-olah dia sendiri di komputer itu.



# Checkpoint 3

## Docker – Cek pemahaman

1. Apa perbedaan image dan container?
2. Kenapa container lebih ringan dari VM?
3. Dua mekanisme OS apa yang membuat container bisa isolasi?

### Jawaban Singkat

1. Image = template read-only; Container = instance jalan dari image.
2. Container pakai kernel host bersama, tidak perlu Guest OS sendiri.
3. Namespaces (pemisahan tampilan) dan Cgroups (pembatasan resource).

# **RAID & Storage Redundancy**

# Masalah: Satu Disk Rusak, Data Hilang

Kenapa kita butuh redundansi penyimpanan?

## Masalah Nyata

- Hard disk memiliki *MTBF* (Mean Time Before Failure) – rata-rata 3–5 tahun.
- Dalam data center dengan 1.000 disk, satu disk rusak hampir setiap minggu.
- Satu disk rusak → semua data di disk itu lenyap.
- Solusi naif: backup manual – lambat dan tidak real-time.

## Analogi: Dua Motor

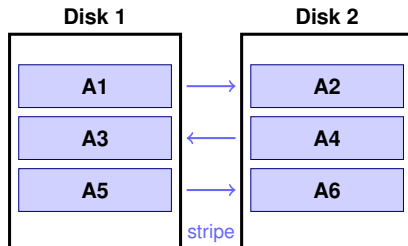
- Bayangkan kita pergi naik motor. Ban motor bisa bocor kapan saja.
- Tanpa cadangan: kita berhenti total dan harus panggil tukang tambal.
- Dengan ban cadangan: kita ganti dan lanjut perjalanan.
- **RAID** adalah “ban cadangan” untuk hard disk – jika satu rusak, yang lain langsung menggantikan.

# RAID 0: Striping (1/2)

Data dipecah dan disebar ke beberapa disk

## Cara Kerja

- Data dipecah (*striped*) menjadi potongan-potongan kecil.
- Potongan disebar ke beberapa disk secara bergantian.
- Membaca/menulis bisa paralel → lebih cepat.
- Dibutuhkan minimal 2 disk.



# RAID 0: Striping (2/2)

Risiko dan kapan sebaiknya digunakan

## Risiko

- **Tidak ada redundansi!**
- Jika satu disk rusak → **semua data hilang**.
- Probabilitas failure =  $N \times P$  (lebih besar dari disk tunggal).

## Kapan Dipakai?

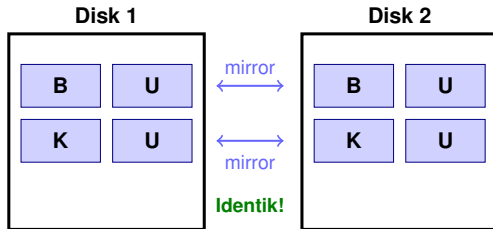
- Video editing (butuh throughput tinggi).
- Data sementara yang bisa di-recreate.

# RAID 1: Mirroring (1/2)

Data ditulis identik ke dua disk

## Cara Kerja

- Setiap data ditulis ke **dua disk** secara identik (*mirror*).
- Jika Disk 1 rusak, Disk 2 masih punya salinan lengkap.
- Baca bisa paralel (lebih cepat), tulis tetap ke dua disk.
- Dibutuhkan minimal 2 disk.



# RAID 1: Mirroring (2/2)

Kekurangan dan kapan sebaiknya digunakan

## Kekurangan

- **Kapasitas terpakai hanya 50%.**
- 2 TB (2 disk  $\times$  1 TB) = hanya 1 TB yang terpakai.

## Kapan Dipakai?

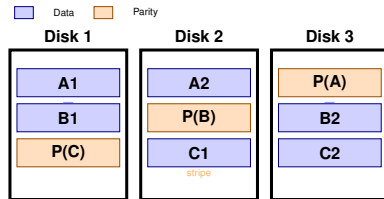
- Sistem kritis (OS, database log).
- Data yang tidak boleh hilang.

# RAID 5: Parity (1/3)

Data + parity disebar ke semua disk

## Cara Kerja RAID 5

- Data + *parity* (informasi pemulihan) disebar ke semua disk.
- Parity memakan kapasitas **1 disk**.
- 3 disk (3 TB) → 2 TB data + 1 TB parity.
- Tahan terhadap **1 disk** rusak.
- Write lebih lambat karena harus hitung parity.



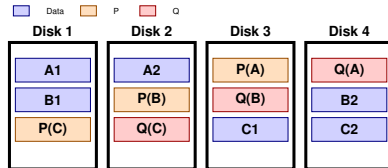


# RAID 6: Parity Ganda (2/3)

Dua parity (P + Q) untuk redundansi ganda

## Cara Kerja RAID 6

- Dua parity (P + Q) – redundansi ganda.
- Parity memakan kapasitas **2 disk**.
- Tahan terhadap **2 disk** rusak **bersamaan**.
- Cocok untuk disk besar dengan rebuild time lama.



# RAID 5 & 6: Analogi Parity (3/3)

Bagaimana parity bekerja?

## Analogi: Permainan Teka-Teki

- Parity seperti **petunjuk** dalam teka-teki silang.
- Jika satu kotak hilang, kita masih bisa menebak isinya dari petunjuk baris dan kolom yang lain.
- **RAID 5** = satu petunjuk – satu kotak hilang masih tertebak.
- **RAID 6** = dua petunjuk – dua kotak hilang pun masih tertebak.

## Mengapa Parity Penting?

- RAID 5 hanya butuh **1 disk ekstra** untuk mengamankan **banyak disk**.
- RAID 6 butuh **2 disk ekstra** tapi tahan 2 kerusakan.
- Lebih efisien dari RAID 1 (50%) untuk data besar.
- Kekurangan: write lebih lambat karena harus hitung parity.

# Perbandingan RAID Levels

| Aspek        | RAID 0       | RAID 1        | RAID 5      | RAID 6           |
|--------------|--------------|---------------|-------------|------------------|
| Minimal disk | 2            | 2             | 3           | 4                |
| Redundansi   | Tidak ada    | Salinan penuh | Parity (1)  | Parity ganda (2) |
| Tahan rusak  | 0 disk       | 1 disk        | 1 disk      | 2 disk           |
| Kapasitas    | 100%         | 50%           | $(N - 1)/N$ | $(N - 2)/N$      |
| Baca         | Sangat cepat | Cepat         | Cepat       | Cepat            |
| Tulis        | Sangat cepat | Sedang        | Lambat      | Paling lambat    |
| Biaya        | Murah        | Mahal         | Sedang      | Sedang           |

## Tips Memilih

- Prioritaskan kecepatan? → RAID 0 (tapi backup terpisah wajib!)
- Prioritaskan keamanan? → RAID 1 (data kecil) atau RAID 5/6 (data besar)

## Checkpoint 4

### RAID – Cek pemahaman

1. Bedanya RAID 0 dan RAID 1?
2. Kenapa RAID 5 lebih efisien kapasitasnya daripada RAID 1?
3. Kapan kita butuh RAID 6 daripada RAID 5?

### Jawaban Singkat

1. RAID 0 = striping (cepat, tanpa redundansi); RAID 1 = mirroring (aman, kapasitas setengah).
2. RAID 5 hanya kehilangan kapasitas 1 disk untuk parity, bukan setengah dari total.
3. Saat pakai disk besar (recovery lama) – dua disk bisa rusak dalam waktu pemulihan.

# Rangkuman Hari Ini

1. **Hak Akses:** Unix membagi izin r/w/x untuk Owner, Group, Others. Ditambah special bits SUID/SGID/Sticky.
2. **Keamanan OS:** Malware (virus, worm, trojan, ransomware) dan mekanisme proteksi OS (user/kernel mode, ASLR, NX bit, isolasi address space).
3. **Docker:** Container sebagai OS-level virtualization yang ringan. Namespace dan Cgroups sebagai kunci isolasi.
4. **RAID:** Redundansi penyimpanan lewat striping (RAID 0), mirroring (RAID 1), atau parity (RAID 5/6).

## Pesan Penutup

Materi pekan depan: **Quiz!** Persiapkan diri dari Week 1–13.

# Ada Pertanyaan?

Selamat belajar dan tetap aman di dunia maya!

*“The only secure system is the one that is powered off, sheathed in a block of concrete and sealed in a lead-lined room with armed guards.”*

— Gene Spafford